

Project Executable Files

Date	4 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	3 Marks

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes/No/NA)
1.	Complete source code (all files and folders)	Yes
2.	README / Setup Guide (instructions to run the project)	Yes
3.	requirements.txt / package.json / dependency file	Yes
4.	Database schema / seed files	NA — no database used; local JSON files (history.json, feedback.json) instead, per Solution Requirements (Folder 2)
5.	Environment configuration file (.env.example)	NA — no external API keys required; Wikipedia's REST API and the local Hugging Face models used here don't need authentication, so config.py holds only non-secret values
6.	Deployed application URL	NA — local deployment only, per Technology Stack (Folder 2)
7.	APK / Executable binary	NA — not a mobile/desktop app No — not built; out of scope for the 5-day local-demo timeline
8.	Dockerfile / Containerization config	No — not built; out of scope for the 5-day local-demo timeline
9.	Test files and test results	Yes — full pytest suite passing (5/5 tests, covering all three services and both API routes)
10.	Demo video or walkthrough	Pending — not yet recorded

Step 2: File / Folder Structure

Personalized Networking Assistant/

- 1. Brainstorming and Ideation/
- 2. Requirement Analysis/
- 3. Project Design Phase/
- 4. Project Planning Phase/
- 5. Project Development Phase/
- 6. Project Testing/
- 7. Project Documentation/
- 8. Project Demonstration/
- app/
 - models/
 - schemas.py — Pydantic request/response models
 - routers/
 - conversation.py — API route definitions
 - services/
 - event_analyzer.py — DistilBERT theme extraction
 - fact_checker.py — Wikipedia fact verification
 - topic_generator.py — GPT-2 starter generation
 - history_logger.py — Session history logging
 - feedback_logger.py — Feedback logging
 - config.py — Model names, Wikipedia API endpoint
 - main.py — FastAPI application entry point
- app/
 - architecture_diagram.jpg
- frontend/
 - streamlit_app.py — Streamlit UI
- tests/ — pytest test files
- venv/ — virtual environment (not committed, in .gitignore)
- requirements.txt — Python dependencies
- history.json — session history log
- feedback.json — feedback log
- .gitignore
- README.md

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	NA — local deployment only
Login Credentials (Demo)	NA — no authentication implemented, per Solution Requirements
Platform / Hosting Provider	NA — runs locally via uvicorn and streamlit run
Repository Link	https://github.com/Juveriyah-Rafiquee/Personalized-Networking-Assistant
Demo Video Link	[Pending — record before final submission]

Step 4: Run Instructions

Pre-requisites: Python 3.11+ (3.13 also works), pip, Git

Step 1: Clone the repository — git clone

<https://github.com/Juveriyah-Rafiquee/Personalized-Networking-Assistant.git>

Step 2: Create a virtual environment — python -m venv venv

Step 3: Activate it — venv\Scripts\Activate.ps1 (Windows) or source venv/bin/activate (Mac/Linux)

Step 4: Install dependencies — pip install -r requirements.txt

Step 5: Start the backend (Terminal 1) — uvicorn app.main:app --reload

Step 6: Start the frontend (Terminal 2, with venv activated) — streamlit run frontend/streamlit_app.py

Step 7: Open the app — Streamlit opens automatically at <http://localhost:8501>;

backend API docs available at <http://127.0.0.1:8000/docs>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1.	GPT-2 Small (not fine-tuned) sometimes produces short or generic conversation starters	Acceptable for capstone scope; documented as a known base-model limitation, verified through direct testing
2.	/generate-conversation response time is 10–20 seconds due to CPU-only sequential inference of two transformer models	Acceptable for a local single-user demo; GPU inference or parallelized calls would improve this in a production setting